# Reinforcement Learning

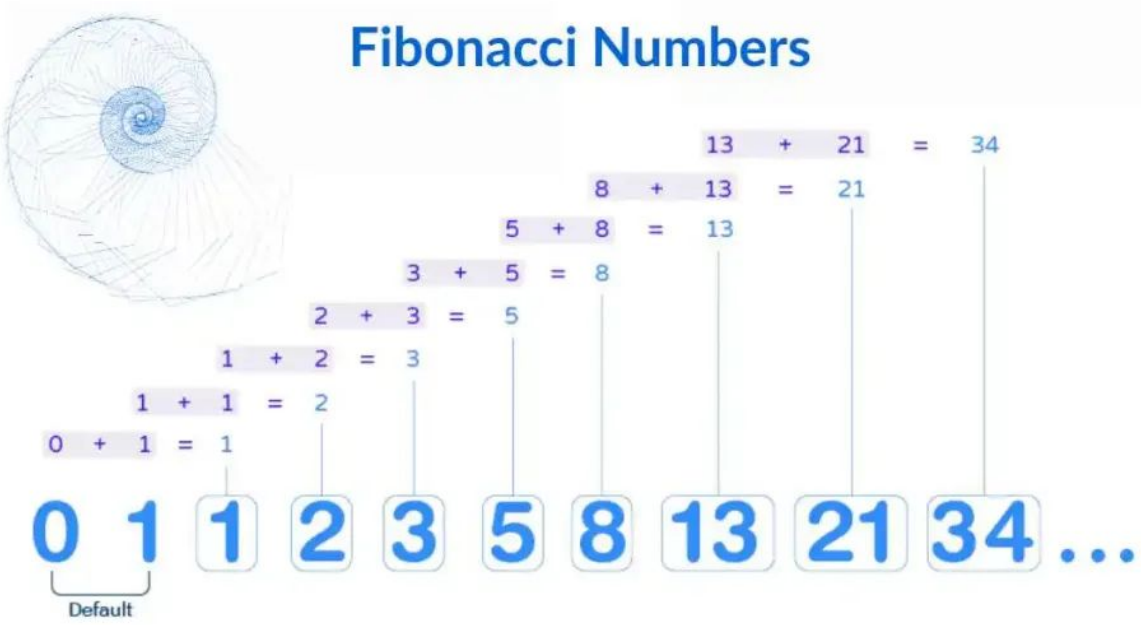
An Introduction

# Course Content

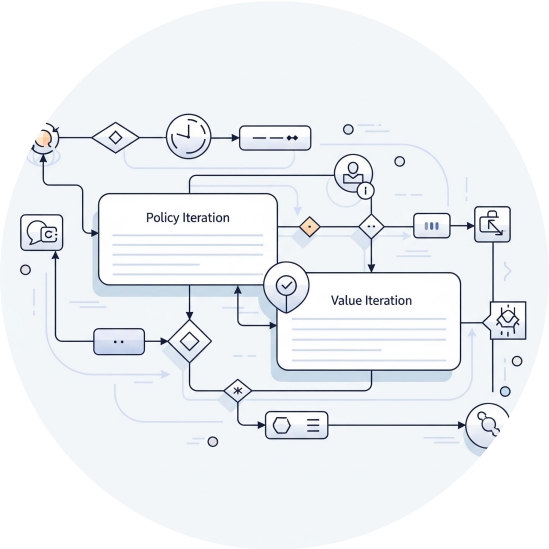
1. Introduction to Reinforcement Learning
2. Markov Decision Processes (MDPs)
3. Policy and Value Functions
4. **Dynamic Programming (DP)**
5. Model-Free methods
6. Value Function Approximation
7. Policy-Gradient and Actor-Critic Methods
8. Deep RL
9. TP Project

# Dynamic Programming (DP)

# What Is Dynamic Programming ?



# Why using it in RL ?




- Dynamic Programming **solves Markov Decision Processes**.
- It requires complete knowledge of the MDP model.
- DP is utilized for planning within the MDP framework.
- DP methods iteratively improve estimates of value functions.
- Key DP techniques include
  - **Policy Iteration**
  - **Value Iteration.**

# Dynamic Programming for MDPs

Dynamic programming requires complete knowledge of the MDP and is utilized for planning within an MDP framework. We can apply it in two setups

- **For prediction:**
  - **Input:** An MDP ***⟨S,A,P,R,γ⟩*** along with a given policy ***π***
  - **Output:** The value function $v_\pi$
- **For control:**
  - **Input:** An MDP ***⟨S,A,P,R,γ⟩***
  - **Output:** The optimal value function $v_*$ and the optimal policy $\pi_*$

# Policy Evaluation vs. Control

Policy Evaluation

$$\pi \longrightarrow v_\pi$$

$$v_\pi(s) = \mathbb{E}_\pi \left[ \overbrace{\boxed{\sum_{k=0}^{\infty} \gamma^k R_{t+k+1}}}^{G_t} \mid S_t = s \right]$$

- **Recall:**

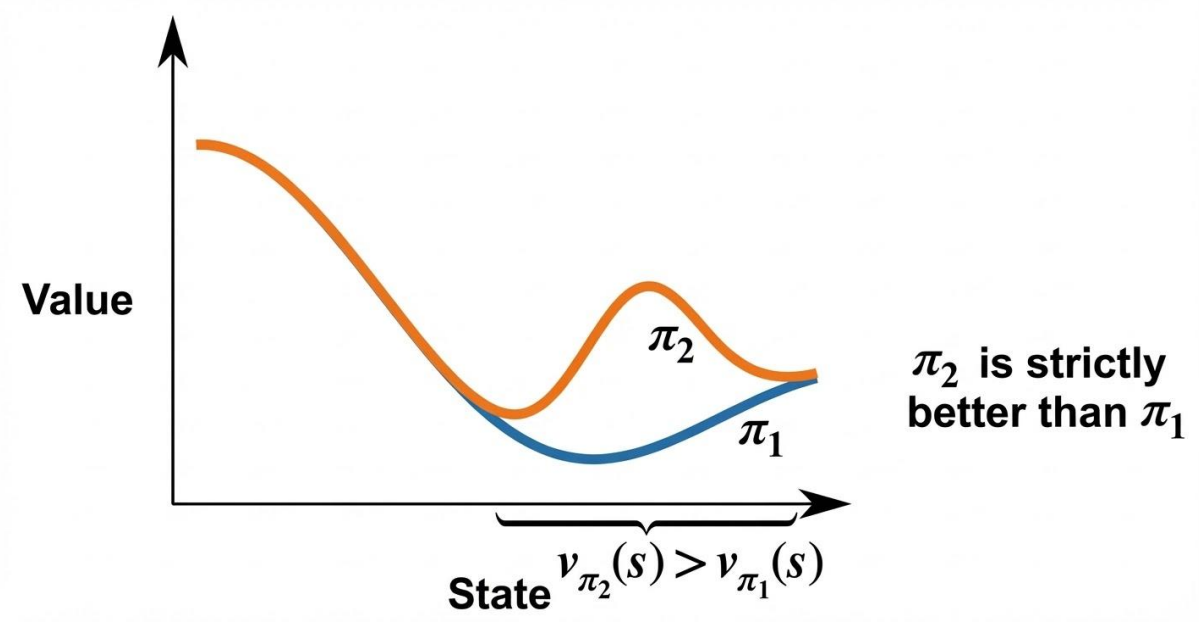
$$v_{\boxed{\pi}}(s) = \sum_{a \in A} \boxed{\pi}(a|s) \sum_{s' \in S} P(s'|s,a) \left[ R(s,a,s') + \gamma v_{\boxed{\pi}}(s') \right]$$

$$\pi, \mathbf{P}, \boldsymbol{\gamma} \longrightarrow \boxed{\textbf{Dynamic Programming}} \longrightarrow v_\pi$$

# Policy Evaluation vs. Control

Control

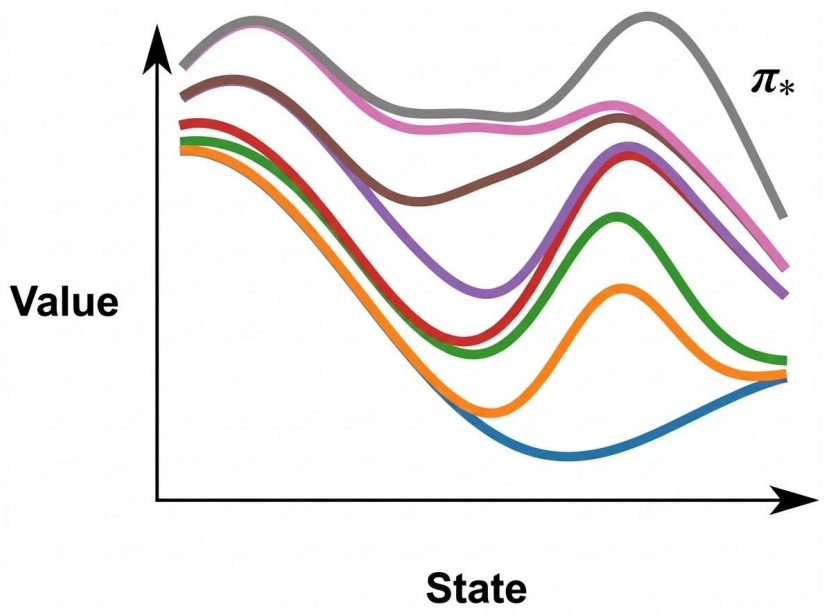
- It is the task that improves the policy



# Policy Evaluation vs. Control

Control

AFTER MULTIPLE ITERATIONS

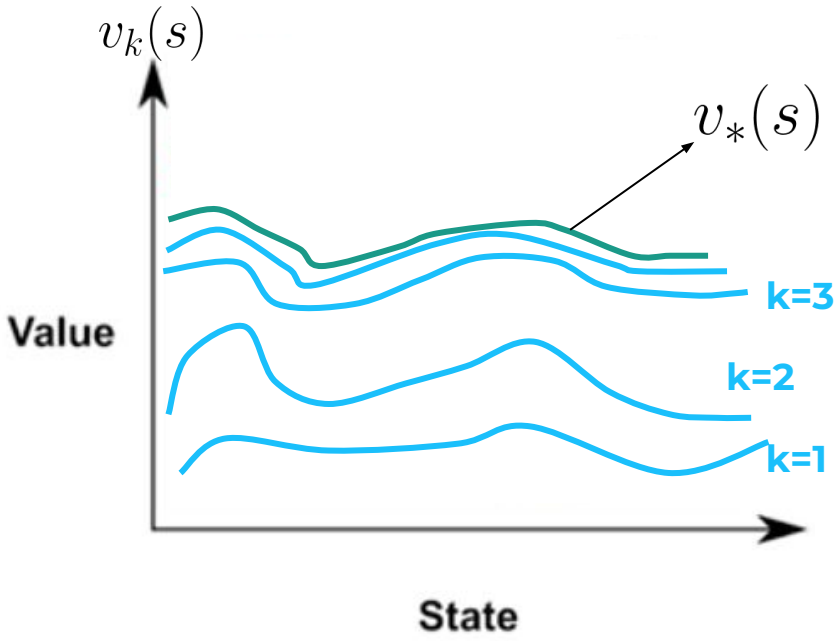


# Iterative Policy Evaluation

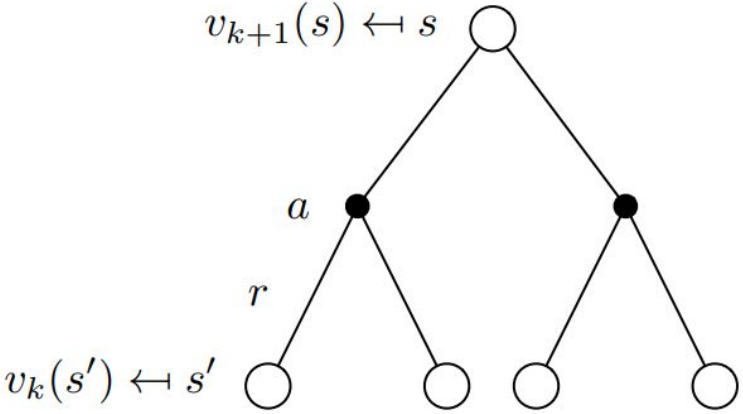
Your First “RL” Algorithm !

# Iterative Policy Evaluation

- **Objective:** Improve the estimate of the value function $v_k(s)$



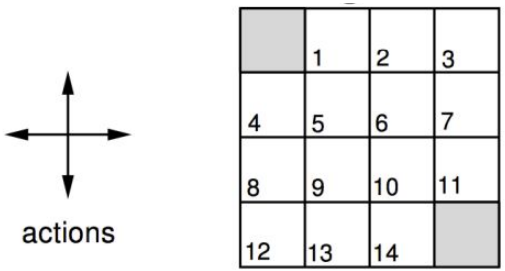
# Iterative Policy Evaluation




$$v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left( \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right)$$

$$\mathbf{v}^{k+1} = \boldsymbol{\mathcal{R}}^{\boldsymbol{\pi}} + \gamma \boldsymbol{\mathcal{P}}^{\boldsymbol{\pi}} \mathbf{v}^k$$

# Iterative Policy Evaluation [1]




$r = -1$
on all transitions

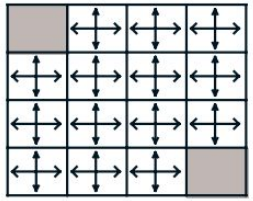
- Undiscounted episodic MDP ($\gamma = 1$)
- Nonterminal states $1, ..., 14$
- One terminal state (shown twice as shaded squares)
- Actions leading out of the grid leave state unchanged
- Reward is $-1$ until the terminal state is reached
- Agent follows uniform random policy

$$\pi(n|\cdot) = \pi(e|\cdot) = \pi(s|\cdot) = \pi(w|\cdot) = 0.25$$

# Iterative Policy Evaluation [1]

| | $v_k$ for the Random Policy | Greedy Policy w.r.t. $v_k$ |
|---|---|---|
| $k = 0$ | | |

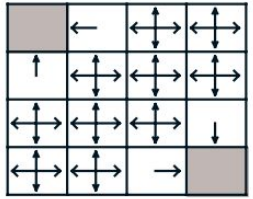
| | | | |
|---|---|---|---|
| 0.0 | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | 0.0 |



random policy

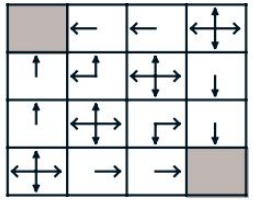
$k = 1$

| | | | |
|---|---|---|---|
| 0.0 | -1.0 | -1.0 | -1.0 |
| -1.0 | -1.0 | -1.0 | -1.0 |
| -1.0 | -1.0 | -1.0 | -1.0 |
| -1.0 | -1.0 | -1.0 | 0.0 |



$k = 2$

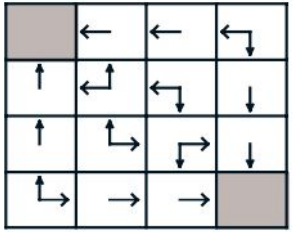
| | | | |
|---|---|---|---|
| 0.0 | -1.7 | -2.0 | -2.0 |
| -1.7 | -2.0 | -2.0 | -2.0 |
| -2.0 | -2.0 | -2.0 | -1.7 |
| -2.0 | -2.0 | -1.7 | 0.0 |



# Iterative Policy Evaluation [1]

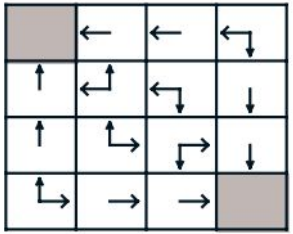
$k = 3$

| | | | |
|---|---|---|---|
| 0.0 | -2.4 | -2.9 | -3.0 |
| -2.4 | -2.9 | -3.0 | -2.9 |
| -2.9 | -3.0 | -2.9 | -2.4 |
| -3.0 | -2.9 | -2.4 | 0.0 |



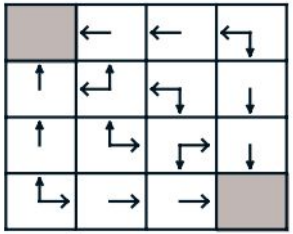
$k = 10$

| | | | |
|---|---|---|---|
| 0.0 | -6.1 | -8.4 | -9.0 |
| -6.1 | -7.7 | -8.4 | -8.4 |
| -8.4 | -8.4 | -7.7 | -6.1 |
| -9.0 | -8.4 | -6.1 | 0.0 |



$k = \infty$

| | | | |
|---|---|---|---|
| 0.0 | -14. | -20. | -22. |
| -14. | -18. | -20. | -20. |
| -20. | -20. | -18. | -14. |
| -22. | -20. | -14. | 0.0 |



optimal policy

# Iterative Policy Evaluation

**Iterative Policy Evaluation, for estimating $V \approx v_\pi$**

Input $\pi$, the policy to be evaluated
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
  $\Delta \leftarrow 0$
  Loop for each $s \in \mathcal{S}$:
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s', r \,|\, s, a)\big[r + \gamma V(s')\big]$
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

# Policy Iteration

Now we're learning Policies

# Policy Iteration

Greedy action

This is achieved in **2 steps:**

1. **Policy Evaluation:**

$$v_\pi(s) = \sum_{a \in A} \pi(a|s) \sum_{s' \in S} P(s'|s,a) \left[ R(s,a,s') + \gamma v_\pi(s') \right]$$

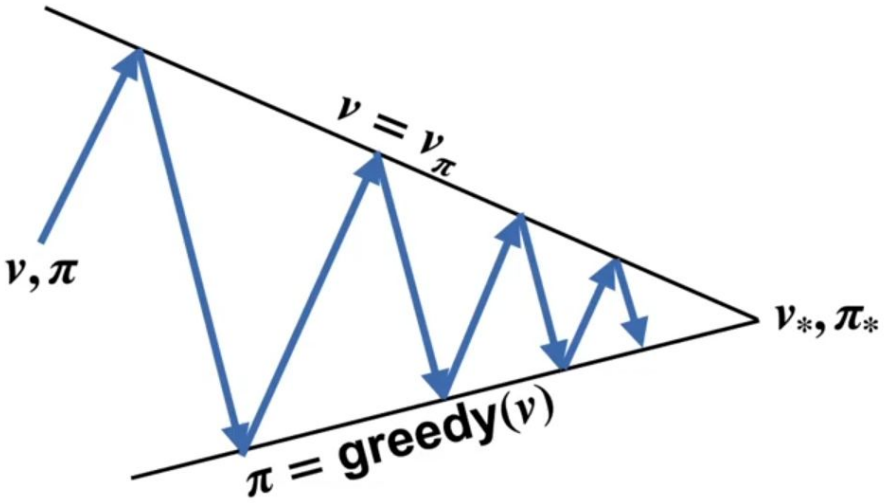
2. **Policy Improvement (update):** considering the greedy action as the best action

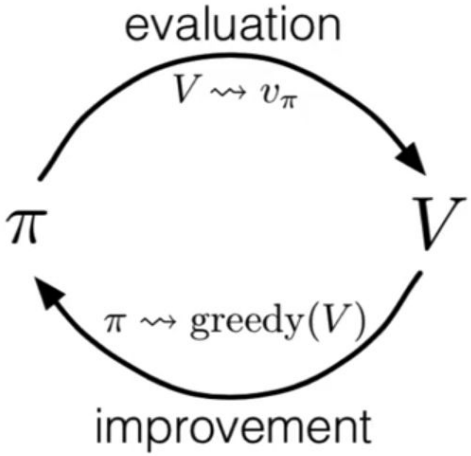
$$\pi'(s) = \underset{a}{argmax}\ v_\pi(s)$$

# Policy Iteration

Each new value function enables us to improve the policy

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*$$



# Policy Iteration

**Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$**

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Loop:
   - $\Delta \leftarrow 0$
   - Loop for each $s \in \mathcal{S}$:
     - $v \leftarrow V(s)$
     - $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s)) \big[r + \gamma V(s')\big]$
     - $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   *policy-stable* $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   - *old-action* $\leftarrow \pi(s)$
   - $\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s', r \mid s, a) \big[r + \gamma V(s')\big]$
   - If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*

   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

## Achieve An Optimal Policy

- If the following Bellman equation is satisfied

$$v_\pi(s) = \max_{a \in \mathcal{A}} q_\pi(s, a)$$

- Then $\boldsymbol{\pi}$ is an optimal policy, with

$$v_\pi(s) = v_*(s) \text{ for all } s \in \mathcal{S}$$

# Example: Policy Iteration

# Policy Iteration

Recall

**Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$**

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Loop:
   - $\Delta \leftarrow 0$
   - Loop for each $s \in \mathcal{S}$:
     - $v \leftarrow V(s)$
     - $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s)) \big[r + \gamma V(s')\big]$
     - $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

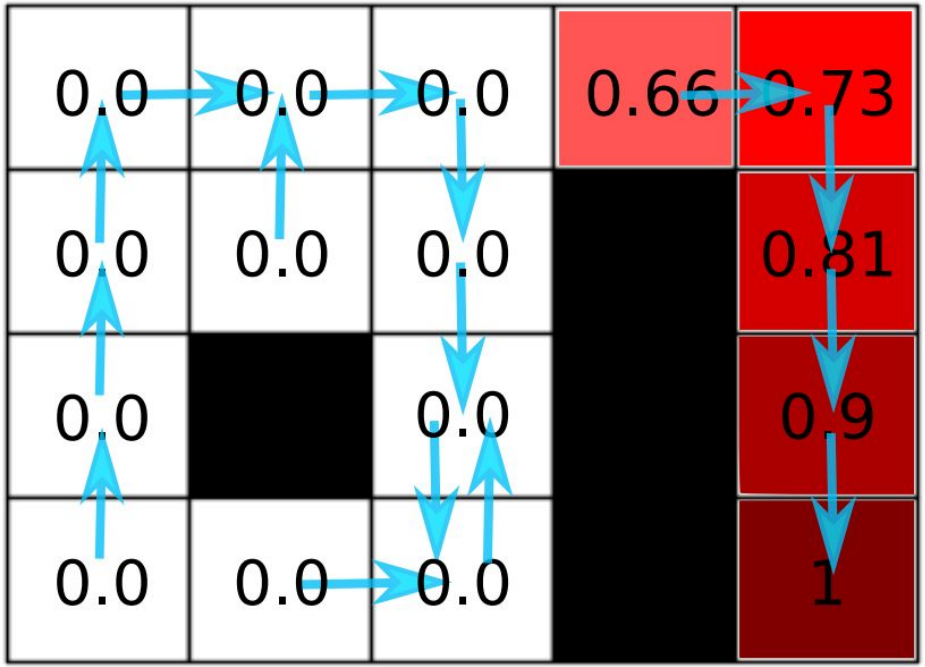
3. Policy Improvement
   *policy-stable* $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   - *old-action* $\leftarrow \pi(s)$
   - $\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s', r \mid s, a) \big[r + \gamma V(s')\big]$
   - If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*

   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

# Policy Iteration

Policy Evaluation

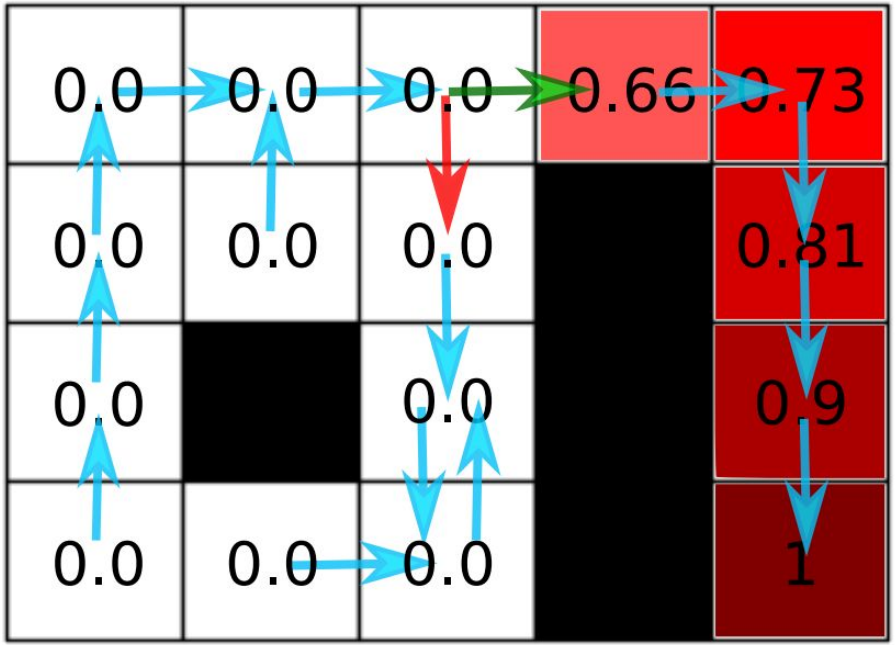
$$\forall s \in S,\ V_i(s) \leftarrow \text{evaluate}(\pi_i(s))$$



# Policy Iteration

Policy Improvement

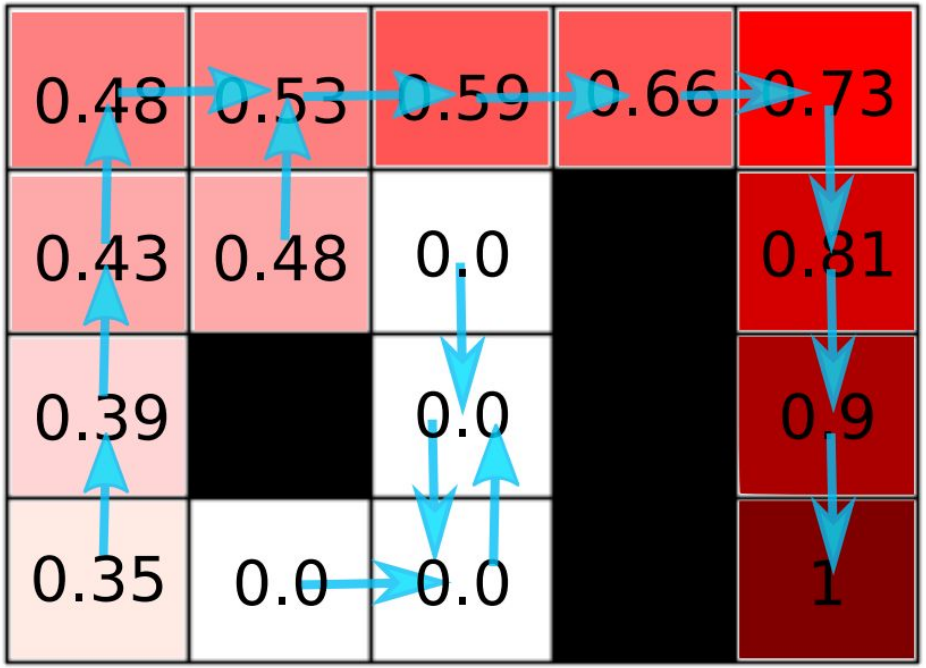
$$\forall s \in S,\ \pi_{i+1}(s) \leftarrow \text{improve}(\pi_i(s), V_i(s))$$



# Policy Iteration

Policy Evaluation

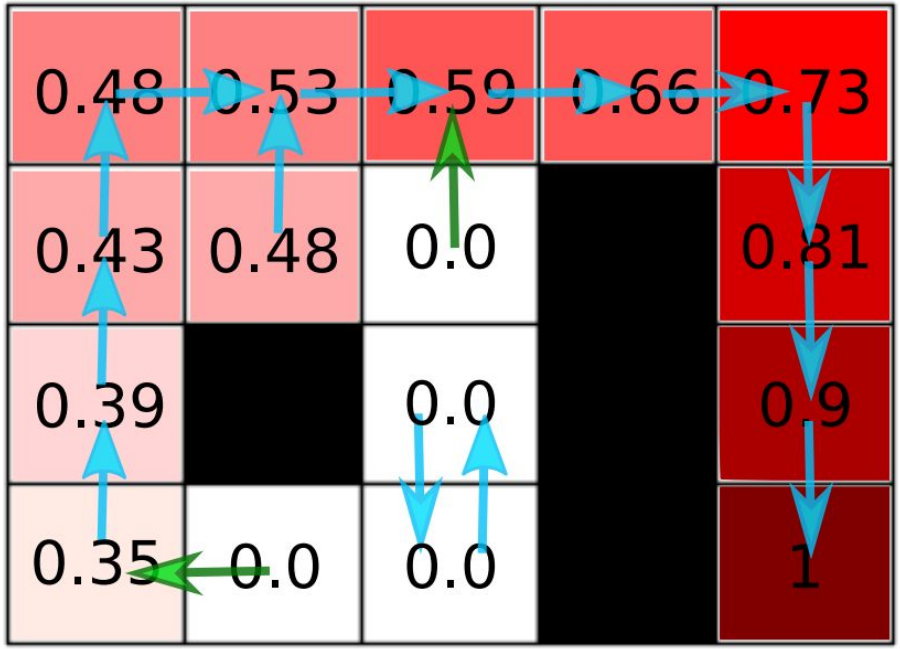
$$\forall s \in S,\ V_i(s) \leftarrow \text{evaluate}(\pi_i(s))$$



# Policy Iteration

Policy Improvement

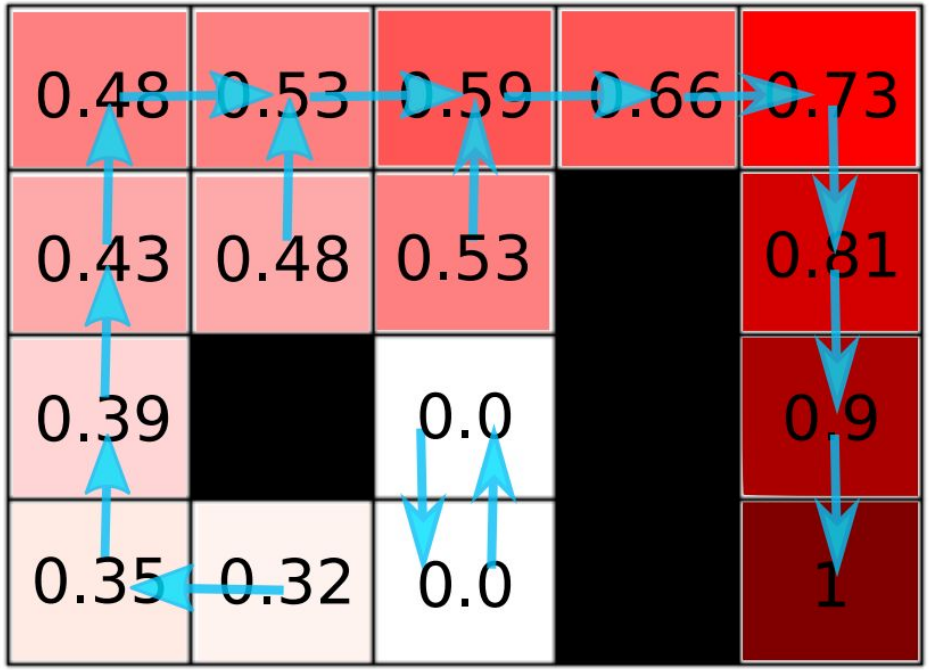
$$\forall s \in S,\ \pi_{i+1}(s) \leftarrow \text{improve}(\pi_i(s), V_i(s))$$



# Policy Iteration

Policy Evaluation

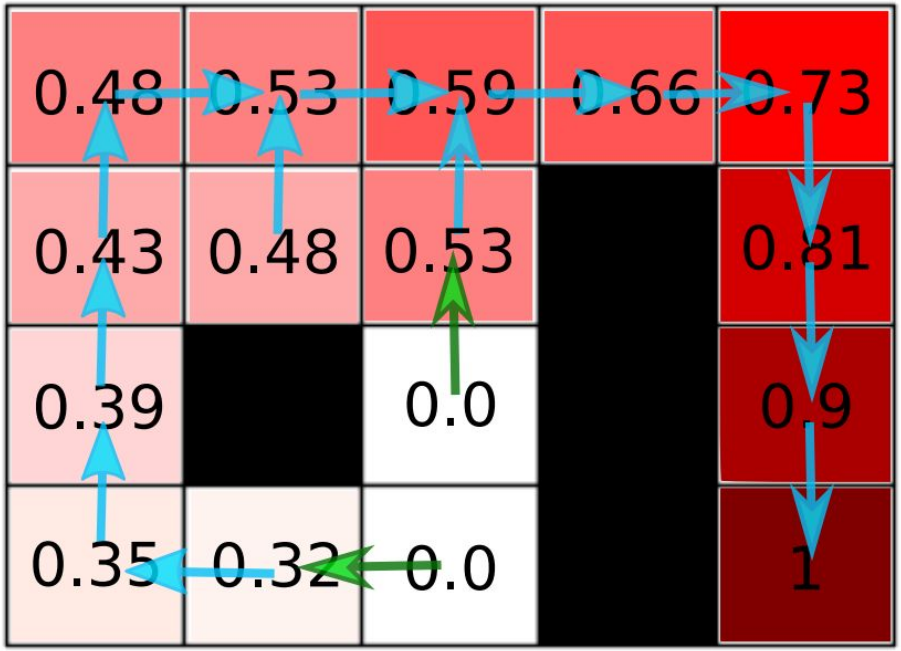
$$\forall s \in S,\ V_i(s) \leftarrow \text{evaluate}(\pi_i(s))$$



# Policy Iteration

Policy Improvement

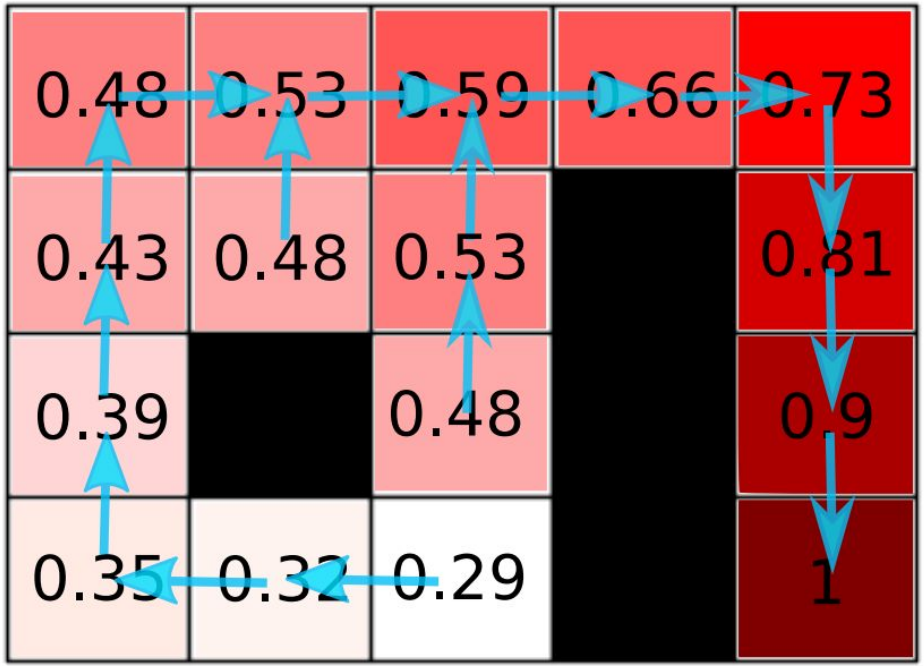
$$\forall s \in S,\ \pi_{i+1}(s) \leftarrow \text{improve}(\pi_i(s), V_i(s))$$



# Policy Iteration

Policy Evaluation

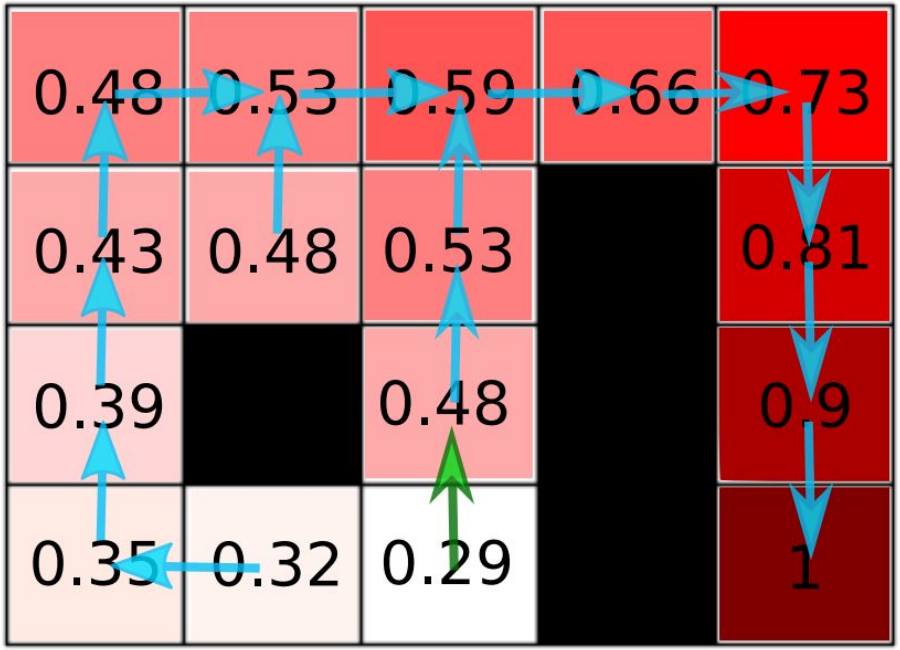
$$\forall s \in S,\ V_i(s) \leftarrow \text{evaluate}(\pi_i(s))$$



# Policy Iteration

Policy Improvement

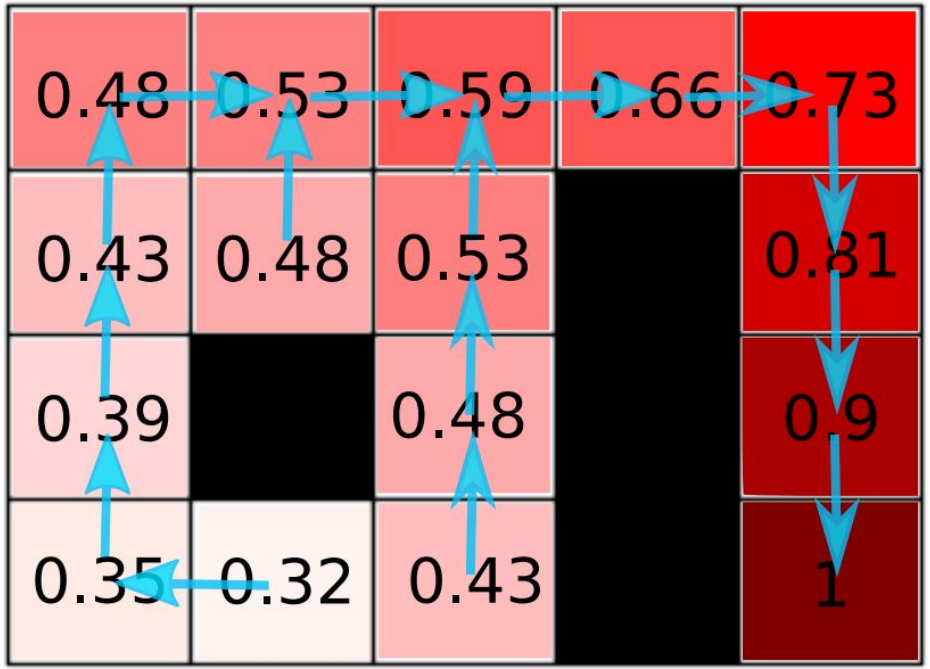
$$\forall s \in S,\ \pi_{i+1}(s) \leftarrow \text{improve}(\pi_i(s), V_i(s))$$



# Policy Iteration

Policy Evaluation

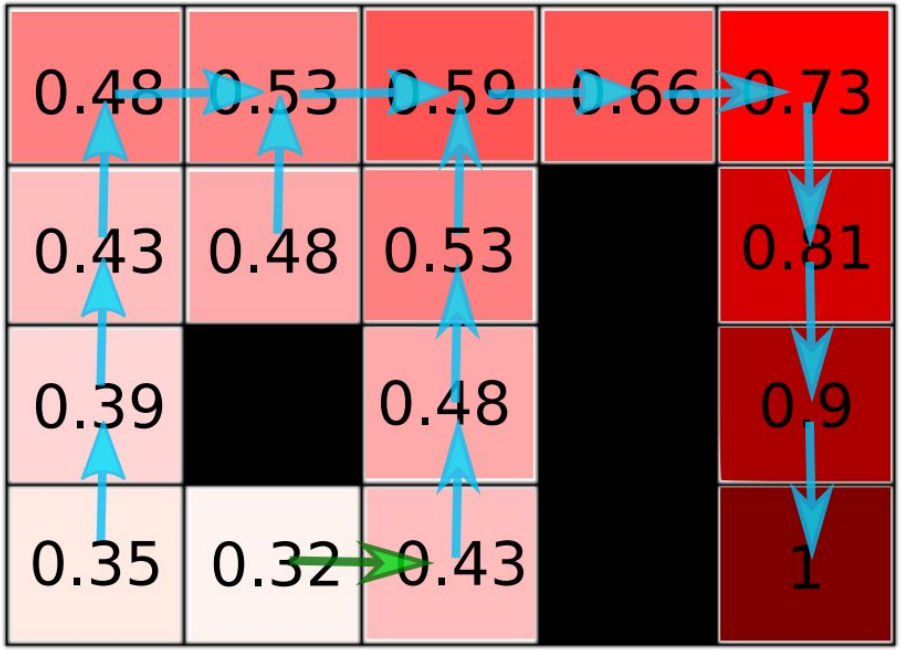
$$\forall s \in S,\ V_i(s) \leftarrow \text{evaluate}(\pi_i(s))$$



# Policy Iteration

Policy Improvement

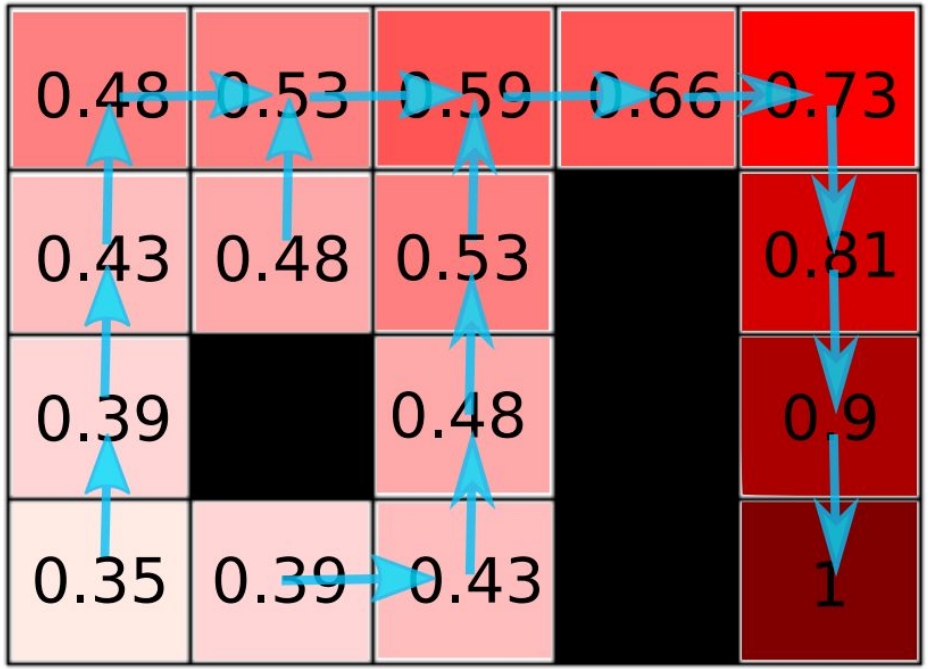
$$\forall s \in S,\ \pi_{i+1}(s) \leftarrow \text{improve}(\pi_i(s), V_i(s))$$



# Policy Iteration

Policy Evaluation

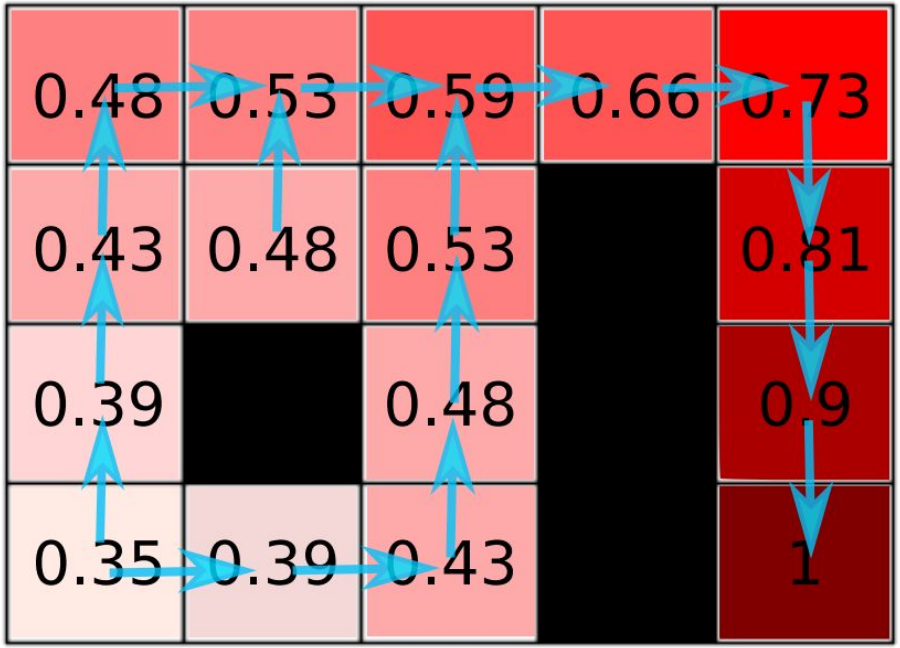
$$\forall s \in S,\ V_i(s) \leftarrow \text{evaluate}(\pi_i(s))$$



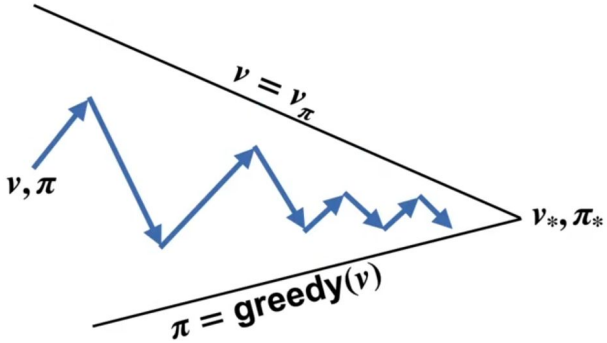
# Policy Iteration

Policy Improvement

$$\forall s \in S,\ \pi_{i+1}(s) \leftarrow \text{improve}(\pi_i(s), V_i(s))$$



# Generalized Policy Iteration




## When should iterations stop?

$k = 3$

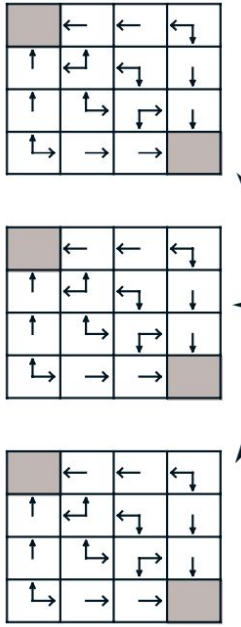
| | | | |
|---|---|---|---|
| 0.0 | -2.4 | -2.9 | -3.0 |
| -2.4 | -2.9 | -3.0 | -2.9 |
| -2.9 | -3.0 | -2.9 | -2.4 |
| -3.0 | -2.9 | -2.4 | 0.0 |

$k = 10$

| | | | |
|---|---|---|---|
| 0.0 | -6.1 | -8.4 | -9.0 |
| -6.1 | -7.7 | -8.4 | -8.4 |
| -8.4 | -8.4 | -7.7 | -6.1 |
| -9.0 | -8.4 | -6.1 | 0.0 |

$k = \infty$

| | | | |
|---|---|---|---|
| 0.0 | -14. | -20. | -22. |
| -14. | -18. | -20. | -20. |
| -20. | -20. | -18. | -14. |
| -22. | -20. | -14. | 0.0 |



optimal policy

# Value Iteration

We can do better than that…

# Value Iteration - principle

- Policy Iteration is heavy
- Wait, we don’t need to use the policy at all..

$$V(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma V(s')]$$

# Use Value Iteration instead of Iterative Policy Iteration

**Value Iteration, for estimating $\pi \approx \pi_*$**

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $v \leftarrow V(s)$
| $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)\big[r + \gamma V(s')\big]$
| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a)\big[r + \gamma V(s')\big]$

# Example: Value Iteration

# Value Iteration

Recall

**Value Iteration, for estimating $\pi \approx \pi_*$**

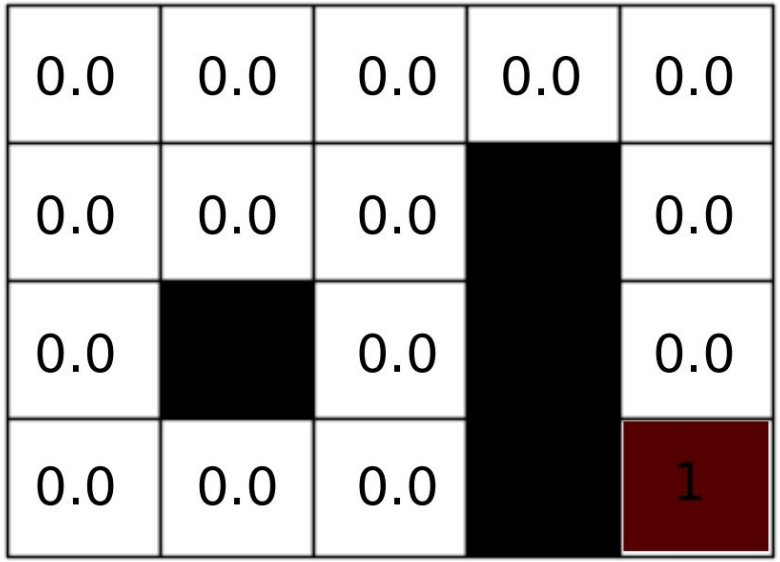
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $v \leftarrow V(s)$
| $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)\big[r + \gamma V(s')\big]$
| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a)\big[r + \gamma V(s')\big]$

$$\sum_{s',r} p(s',r|s,a)[r + \gamma V(s')] = r + \gamma \sum_{s'} p(s'|s,a)V(s')$$

# Value Iteration




$$\forall s \in S,\ V_{i+1}(s) \leftarrow \max_{a \in A} \left[ r(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V_i(s') \right]$$

# Value Iteration

| | | | | |
|---|---|---|---|---|
| 0.0 | 0.0 | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | | 0.0 |
| 0.0 | | 0.0 | | 0.9 |
| 0.0 | 0.0 | 0.0 | | 1 |

$$\forall s \in S,\ V_{i+1}(s) \leftarrow \max_{a \in A} \left[ r(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V_i(s') \right]$$

# Value Iteration

| | | | | |
|---|---|---|---|---|
| 0.0 | 0.0 | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | | 0.81 |
| 0.0 | | 0.0 | | 0.9 |
| 0.0 | 0.0 | 0.0 | | 1 |

$$\forall s \in S,\ V_{i+1}(s) \leftarrow \max_{a \in A} \left[ r(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V_i(s') \right]$$

# Value Iteration

| | | | | |
|---|---|---|---|---|
| 0.0 | 0.0 | 0.0 | 0.0 | 0.73 |
| 0.0 | 0.0 | 0.0 | | 0.81 |
| 0.0 | | 0.0 | | 0.9 |
| 0.0 | 0.0 | 0.0 | | 1 |

$$\forall s \in S,\ V_{i+1}(s) \leftarrow \max_{a \in A} \left[ r(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V_i(s') \right]$$

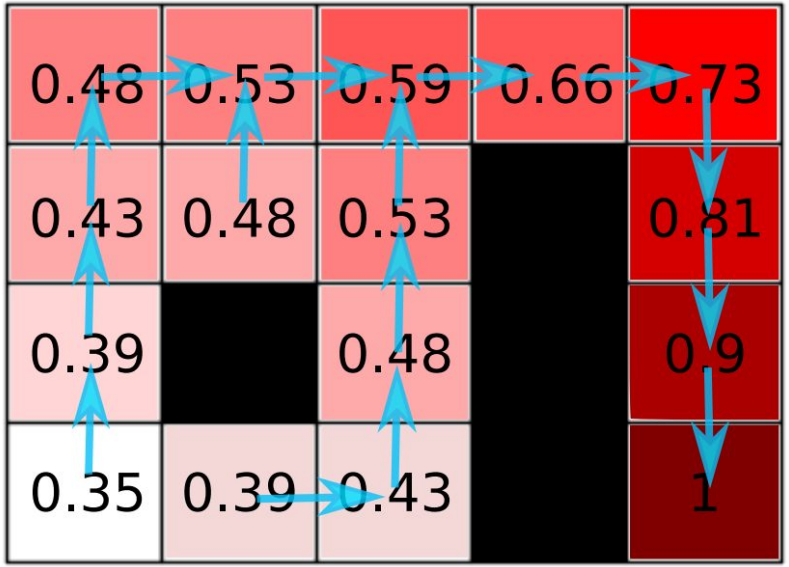
# Value Iteration

| | | | | |
|---|---|---|---|---|
| 0.48 | 0.53 | 0.59 | 0.66 | 0.73 |
| 0.43 | 0.48 | 0.53 | | 0.81 |
| 0.39 | | 0.48 | | 0.9 |
| 0.35 | 0.39 | 0.43 | | 1 |

$$\forall s \in S,\ V_{i+1}(s) \leftarrow \max_{a \in A} \left[ r(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V_i(s') \right]$$

# Value Iteration

- We have iterated on values, and determined a policy out of it (without necessarily representing it if using **Q(s, a)**



# Wrap-Up

Recall - Blue = to know well

| Aspect | Policy Evaluation | Policy Iteration | Value Iteration |
|---|---|---|---|
| **Goal** | Evaluate a fixed policy $\pi$. | Find the optimal policy $\pi^*$. | Find the optimal value function $V^*$. |
| **Process** | Computes $V_\pi(s)$ iteratively. | Alternates between policy evaluation and improvement. | Iteratively updates $V(s)$ using the Bellman optimality equation. |
| **Policy Update** | No updates to the policy. | Updates policy in the improvement step. | Policy is derived after $V(s)$ converges. |
| **Use Case** | Analyze the performance of a specific policy. | Iteratively refine and improve a policy. | Efficiently compute both $V^*$ and $\pi^*$. |
| **Convergence Speed** | Depends on policy evaluation only. | Typically slower than value iteration. | Often faster due to single-step updates. |

# Efficiency of Dynamic Programming

DP algorithms

| Problem | Bellman Equation | Algorithm |
|---|---|---|
| Prediction | Bellman Expectation Equation | Iterative Policy Evaluation |
| Control | Bellman Expectation Equation + Greedy Policy Improvement | Policy Iteration |
| Control | Bellman Optimality Equation | Value Iteration |

# Dynamic Programming

Pros & Cons

- It is **highly effective for solving MDPs** when we **know** the transition probabilities and reward functions.
- It **guarantees convergence** with a complete and accurate model
- It requires a **complete model** of the environment, with **known transition probabilities** and rewards for each state-action pair
- DP algorithms, like Policy Iteration and Value Iteration, often **involve updating the value of every state in each iteration**, which can become **computationally intensive for large state spaces**
- DP methods are **planning** algorithms, meaning they **rely on a fixed model** and **can't naturally handle exploration** of unknown environments

**Next** → **Model-free RL approaches**

# TP Dynamic Programming

2 -dynamic_programming.student.ipynb

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

Copyright © Sorbonne University.

This source code is licensed under the MIT license found in the LICENSE file in the root directory of this source tree.

## Outlook

In this notebook, we study the **value iteration** and **policy iteration** algorithms in a maze environment.

## Installation

```
try:
    from easypip import easyimport
except ModuleNotFoundError:
    from subprocess import run
    assert run(["pip", "install", "easypip"]).returncode == 0, "Could not install easypip"
    from easypip import easyimport

easyimport("swig")
easyimport("bbrl_utils").setup(maze_mdp=True)

from pathlib import Path
from typing import List, Tuple
```